

# 改进后的软件过程定义文档（AI 赋能版）

## 文档信息

- 文档名称：AI 赋能迭代式软件过程定义文档
- 版本号：V2.0
- 编制单位：软件项目组
- 完成日期：2026 年春
- 适用范围：中小型 Web 业务系统开发项目，团队规模 5-15 人，单迭代周期 2-4 周
- 编制依据：《软件过程剪裁与定义》V1.0、《AI 技术应用场景匹配报告》、《软件过程改进方案设计文档》

## 一、过程概述

### 1.1 过程定义与核心理念

本过程是在传统迭代式软件开发过程的基础上，深度融合生成式大模型、智能代码工具、AI 测试平台、AIOps 等智能化技术，重构全生命周期活动分工与协作流程而形成的新型软件过程体系。

过程遵循“人工主导决策、AI 赋能执行”的核心原则：业务逻辑判断、架构设计、质量验收、风险决策等核心创造性工作由人工主导；文档撰写、代码生成、用例设计、规则校验、异常检测等标准化、重复性工作由 AI 辅助完成。通过明确人机分工边界，在不颠覆传统软件工程质量管控逻辑的前提下，最大化释放 AI 技术的效率价值，构建质量可控、效率领先、可持续迭代的人机协同开发范式。

### 1.2 过程目标

- 效率提升目标**：全生命周期核心环节人均产出效率较传统过程提升 30% 以上，文档编写、脚本维护等重复性劳动占比降低 50%，显著缩短项目交付周期。
- 质量保障目标**：需求一致性缺陷、代码安全漏洞、测试覆盖盲区等问题的前置检出率提升 25%，线上故障平均恢复时长缩短 40%，产品交付质量稳定性持续增强。
- 能力沉淀目标**：形成标准化的 AI 工具使用规范、Prompt 模板库、领域知识库与质量校验机制，将个人经验转化为团队可复用的过程能力。
- 风险可控目标**：建立 AI 输出全流程质量管控体系，将数据安全、输出幻觉、技术依赖等风险控制可在接受范围内，保障过程运行稳定可靠。

### 1.3 生命周期划分

本过程采用迭代式开发模式，每个迭代周期完整执行软件全生命周期的五大核心阶段，依次为：需求分析阶段、系统设计阶段、编码实现阶段、测试验证阶段、运维监控阶段。各阶段边界清

晰、交付物明确，AI 技术深度嵌入每个阶段的核心执行节点，形成端到端的智能化赋能链路。

## 二、角色与职责定义

在传统项目角色体系基础上，新增 2 类 AI 专项角色，同时明确所有角色的人机协同职责，形成权责清晰、分工明确的角色体系。

表格

| 角色分类    | 角色名称     | 核心职责                                                                             |
|---------|----------|----------------------------------------------------------------------------------|
| 项目管理角色  | 项目经理     | 负责项目整体进度管控、资源协调、风险管理与跨团队沟通；组织迭代评审与过程改进；审批 AI 工具引入与过程变更                           |
| 技术决策角色  | 系统架构师    | 负责技术选型、系统架构设计、技术方案评审；制定代码规范与技术约束；校验 AI 生成设计方案的技术合理性                              |
| 需求业务角色  | 需求分析师    | 负责用户调研、业务需求梳理与需求变更管理；校验 AI 生成需求文档的业务准确性；主导需求评审                                   |
| 开发实现角色  | 开发工程师    | 负责核心业务逻辑设计与功能编码；校验 AI 生成代码的正确性与可维护性；参与代码评审与功能调试                                  |
| 质量保障角色  | 测试工程师    | 负责制定测试策略与测试计划；校验 AI 生成测试用例的业务合理性；执行核心测试环节与缺陷闭环管理                                 |
| 运维保障角色  | 运维工程师    | 负责部署运维体系搭建、监控指标定义与故障最终处置；校验 AI 根因分析结论的准确性；沉淀运维知识库                                |
| AI 专项角色 | AI 过程赋能师 | 负责 AI 工具的选型、部署与开发工具链集成；制定各阶段 Prompt 规范与 AI 输出质量标准；开展团队 AI 工具使用培训；持续跟踪并优化 AI 应用效果 |
| AI 专项角色 | AI 测试分析师 | 负责 AI 测试工具的业务场景调优；维护测试领域知识库与自动化脚本自愈规则；分析 AI 生成用例的质量数据；推动测试 AI 能力持续迭代             |

## 三、分阶段详细过程定义

### 3.1 需求分析阶段

#### 3.1.1 阶段目标

全面、准确地捕获用户需求，形成标准化、无冲突的需求基线，为后续设计与开发提供清晰、一致的输入，最大限度降低因需求偏差导致的返工成本。

3.1.2 入口准则

- 项目立项完成，业务目标与范围明确；
- 用户调研渠道与对接人确认；
- 需求文档标准模板与 AI 工具配置完成。

3.1.3 核心活动与 AI 参与节点

1. **需求调研与素材归集（人工主导）** 需求分析师通过用户访谈、业务资料梳理、竞品分析等方式开展需求调研，整理形成结构化的访谈纪要、用户故事清单、业务规则说明等原始素材，确保素材完整覆盖业务核心诉求。
2. **AI 生成需求文档初稿（AI 执行）** 需求分析师将整理后的原始素材输入大模型，按照标准需求规格说明书模板配置生成规则，由 AI 自动生成包含功能需求、非功能需求、业务用例、术语定义的需求文档初稿。AI 同步完成文档结构化排版与术语初步统一。
3. **人工业务校验与优化（人工主导）** 需求分析师对 AI 生成的文档初稿进行逐章校验，修正业务逻辑偏差、补充行业专属规则、调整需求优先级与边界定义，确保需求内容完全符合业务实际。
4. **AI 需求一致性与完整性校验（AI 执行）** 调用 AI 需求校验工具，对优化后的需求文档进行全量扫描，自动识别需求条目冲突、术语定义不统一、边界描述模糊、异常场景缺失等问题，输出《需求一致性校验报告》。
5. **需求评审与基线发布（人工主导）** 需求分析师根据校验报告修复问题后，组织开发、测试、架构等角色开展正式需求评审。评审通过后，需求文档纳入基线管理，作为后续阶段的唯一输入依据。

3.1.4 配套 AI 工具与输出

- 工具：私有化部署通用大模型、需求一致性校验工具
- AI 直接输出物：需求文档初稿、需求一致性校验报告

3.1.5 出口准则

- 需求规格说明书通过正式评审并纳入基线；
- 需求点无明显冲突与遗漏，业务口径统一；
- 所有相关角色对需求范围达成一致。

3.1.6 核心交付物

《需求规格说明书》、《需求一致性校验报告》、需求评审记录

## 3.2 系统设计阶段

### 3.2.1 阶段目标

基于需求基线完成系统技术方案设计，明确架构选型、模块划分、接口定义与数据模型，保障系统的可扩展性、可维护性与技术合理性。

### 3.2.2 入口准则

- 需求规格说明书正式基线化；
- 技术栈与架构约束明确。

### 3.2.3 核心活动与 AI 参与节点

1. **架构方案整体设计（人工主导）** 架构师根据需求与技术约束，确定系统整体架构风格、技术栈选型、模块划分原则与核心技术方案，完成架构顶层设计。
2. **AI 辅助设计方案细化与设计模式推荐（AI 辅助）** 架构师输入架构顶层设计与模块需求，由 AI 推荐适配的设计模式、行业通用架构方案与技术实现路径，辅助完成子模块设计方案细化，提供多维度选型参考。
3. **AI 自动生成 UML 模型与接口文档初稿（AI 执行）** 将设计方案与需求文档输入 AI 设计工具，自动生成类图、时序图、组件图等 UML 模型初稿，同时自动生成接口定义文档骨架，包含请求参数、响应结构、错误码等标准字段。
4. **人工设计优化与细节完善（人工主导）** 架构师与设计师对 AI 生成的模型与文档进行校验修正，补充业务专属设计细节，优化接口定义与数据模型，形成完整的系统设计方案。
5. **设计评审与基线确认（人工主导）** 组织开展设计评审，重点验证方案的技术可行性、需求匹配度与扩展性，评审通过后系统设计文档纳入基线管理。

### 3.2.4 配套 AI 工具与输出

- 工具：架构设计 AI 辅助工具、UML 自动生成工具
- AI 直接输出物：设计模式推荐报告、UML 模型初稿、接口文档骨架

### 3.2.5 出口准则

- 系统设计文档通过正式评审并基线化；
- 模块划分清晰，接口定义完整，技术方案可落地；
- 设计方案与需求基线完全对齐。

### 3.2.6 核心交付物

《系统架构设计文档》、《详细设计说明书》、UML 模型文件、接口设计文档

## 3.3 编码实现阶段

### 3.3.1 阶段目标

按照设计文档完成功能代码开发，保障代码质量符合规范，功能逻辑正确，可进入测试环节。

### 3.3.2 入口准则

- 详细设计文档基线化；
- 开发环境与代码规范配置完成；
- AI 编码与代码审查工具集成就绪。

### 3.3.3 核心活动与 AI 参与节点

1. **任务拆解与开发准备（人工主导）** 项目经理与开发负责人拆解开发任务，明确任务优先级、交付标准与时间节点，开发人员完成开发环境搭建与代码分支创建。
2. **AI 辅助编码与代码生成（人机协同）** 开发人员基于设计文档开展编码工作，借助代码大模型完成样板代码生成、接口实现、数据转换、注释自动生成、代码重构等重复性工作；核心业务逻辑与复杂算法由开发人员自主编写。开发过程中 AI 实时提供代码补全与语法纠错支持。
3. **AI 生成单元测试用例（AI 执行）** 代码编写完成后，AI 自动根据函数逻辑与输入输出规则生成单元测试用例与测试代码，开发人员补充业务场景校验后执行本地单元测试。
4. **本地开发调试与功能自测（人工主导）** 开发人员完成功能调试与本地自测，验证基础功能逻辑正确性，修复明显问题后提交代码。
5. **AI 自动化代码审查（AI 执行）** 代码提交后自动触发 AI 代码审查，工具自动检测代码规范违规、安全漏洞、性能缺陷、常见逻辑错误与坏味道，输出可定位的修改建议与风险等级。
6. **人工代码评审与代码合入（人工主导）** 开发人员修复 AI 检出的问题后，进入人工代码评审环节。评审重点关注架构一致性、业务逻辑正确性、代码可维护性等 AI 无法完全覆盖的内容。评审通过后代码合入主干分支。

### 3.3.4 配套 AI 工具与输出

- 工具：GitHub Copilot、SonarQube AI 代码审查工具
- AI 直接输出物：代码片段与补全建议、单元测试代码、代码审查报告

### 3.3.5 出口准则

- 所有开发任务完成，功能自测通过；
- 代码通过 AI 审查与人工评审；
- 单元测试覆盖率达到既定标准。

### 3.3.6 核心交付物

功能源代码、单元测试用例、代码审查报告、代码评审记录

## 3.4 测试验证阶段

### 3.4.1 阶段目标

全面验证系统功能与非功能特性，发现并闭环所有缺陷，保障交付产品符合需求与质量标准。

### 3.4.2 入口准则

- 代码合入主干，测试版本构建完成；
- 需求基线与设计文档稳定；
- AI 测试工具配置与知识库就绪。

### 3.4.3 核心活动与 AI 参与节点

1. **测试计划与策略制定（人工主导）** 测试工程师根据项目范围与迭代目标制定测试计划，明确测试范围、测试类型、通过标准与进度安排，确定核心风险场景。
2. **AI 生成功能测试用例初稿（AI 执行）** 将需求规格说明书、接口文档导入 AI 测试平台，配置用例颗粒度与场景覆盖规则，由 AI 自动生成功能测试用例，覆盖正常场景、边界场景、异常场景三大类，输出标准化用例模板。
3. **人工用例校验与场景补充（人工主导）** 测试工程师对 AI 生成的用例进行业务合理性校验，删除无效用例，补充复杂业务流程、专属业务规则与高风险场景的测试用例，形成最终可执行的测试用例集。
4. **AI 生成 / 自愈自动化测试脚本（AI 执行）** 新版本迭代时，AI 基于测试用例自动生成接口与 UI 自动化测试脚本；针对历史版本失效的自动化脚本，AI 自动识别页面元素、接口字段变更，自动修正脚本定位逻辑与参数，完成脚本自愈。
5. **测试执行与 AI 辅助缺陷分析（人机协同）** 执行测试任务，测试工程师记录缺陷。AI 辅助对缺陷进行自动分类、优先级标注与初步根因分析，关联缺陷与代码模块，提升缺陷处理效率。
6. **缺陷闭环与测试总结（人工主导）** 开发人员修复缺陷后，测试工程师完成回归验证，确保所有缺陷闭环。测试结束后，测试工程师输出测试报告，总结版本质量与 AI 工具使用效果。

### 3.4.4 配套 AI 工具与输出

- 工具：Testin XAgent 智能测试平台
- AI 直接输出物：测试用例初稿、自动化测试脚本、缺陷初步分析报告

### 3.4.5 出口准则

- 所有测试用例执行完成，缺陷全部闭环；



- 版本达到上线质量标准；
- 测试报告通过评审。

### 3.4.6 核心交付物

测试计划、测试用例集、自动化测试脚本、缺陷管理报告、测试总结报告

## 3.5 运维监控阶段

### 3.5.1 阶段目标

保障系统上线后稳定运行，快速响应与处置故障，持续优化系统性能，实现运维工作从被动响应向主动预测转型。

### 3.5.2 入口准则

- 系统正式上线运行；
- 监控指标体系与数据采集链路搭建完成。

### 3.5.3 核心活动与 AI 参与节点

1. **系统部署与监控体系搭建（人工主导）** 运维工程师完成系统部署上线，搭建指标、日志、调用链全维度监控体系，定义告警阈值与分级规则。
2. **AI 实时异常检测与告警聚合（AI 执行）** AIOps 平台实时采集系统运行数据，基于历史基线进行异常检测，主动识别隐性故障；同时对多源告警进行关联聚合，过滤无效告警与重复告警，输出高优先级故障告警。
3. **AI 自动根因分析与修复建议（AI 执行）** 触发故障告警后，AI 智能体自动关联调用链、日志、指标数据，进行跨组件故障溯源，定位根因节点，输出排查路径与初步修复建议。
4. **人工故障确认与处置（人工主导）** 运维工程师校验 AI 根因分析结论，执行故障修复与业务恢复操作；重大故障按照应急预案开展处置，确保业务影响最小化。
5. **复盘优化与 AI 模型迭代（人机协同）** 故障处置完成后开展复盘，将故障场景、根因、处置方案沉淀至运维知识库，用于优化 AI 检测模型与分析规则，持续提升 AI 运维准确率。

### 3.5.4 配套 AI 工具与输出

- 工具：云原生 AIOps 智能运维平台
- AI 直接输出物：告警聚合结果、异常检测报告、根因分析与修复建议

### 3.5.5 阶段闭环

运维阶段为持续运行的闭环过程，系统下线前持续执行监控 - 告警 - 处置 - 优化的循环，每个迭代周期同步完成运维数据复盘与 AI 能力迭代。

3.5.6 核心交付物

运维监控报告、故障处置记录、根因分析报告、运维知识库

四、全生命周期过程总流程图

---



## 需求分析阶段

需求调研与素材归集

AI生成需求初稿

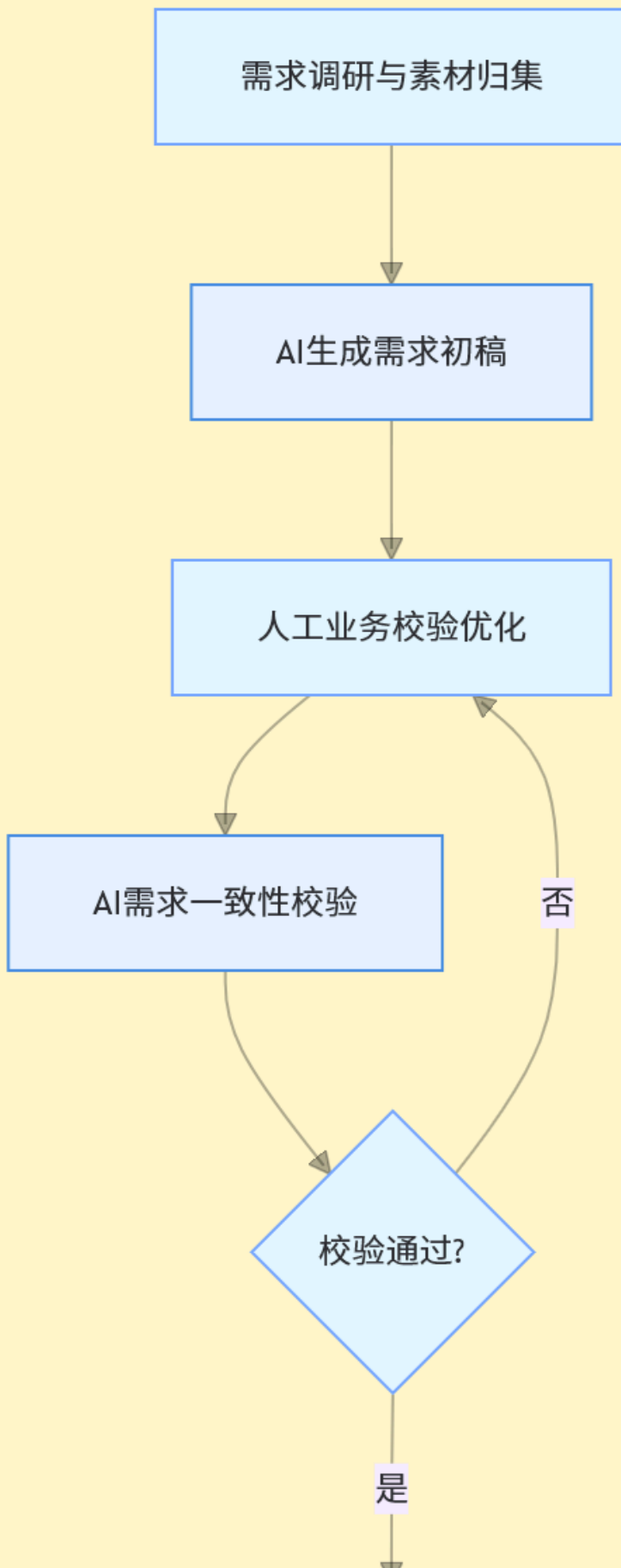
人工业务校验优化

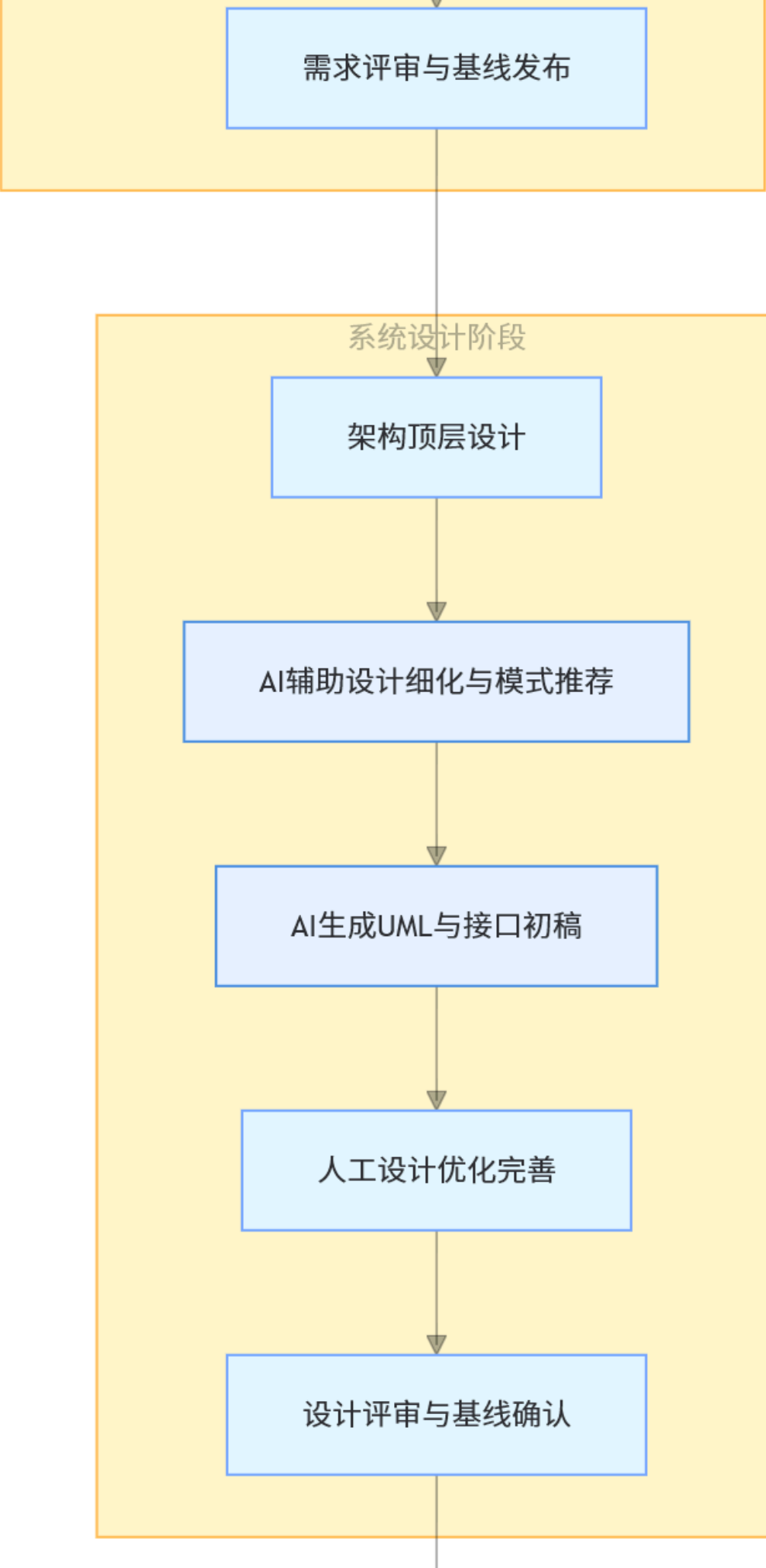
AI需求一致性校验

校验通过?

否

是





编码实现阶段

任务拆解与开发准备

AI辅助编码实现

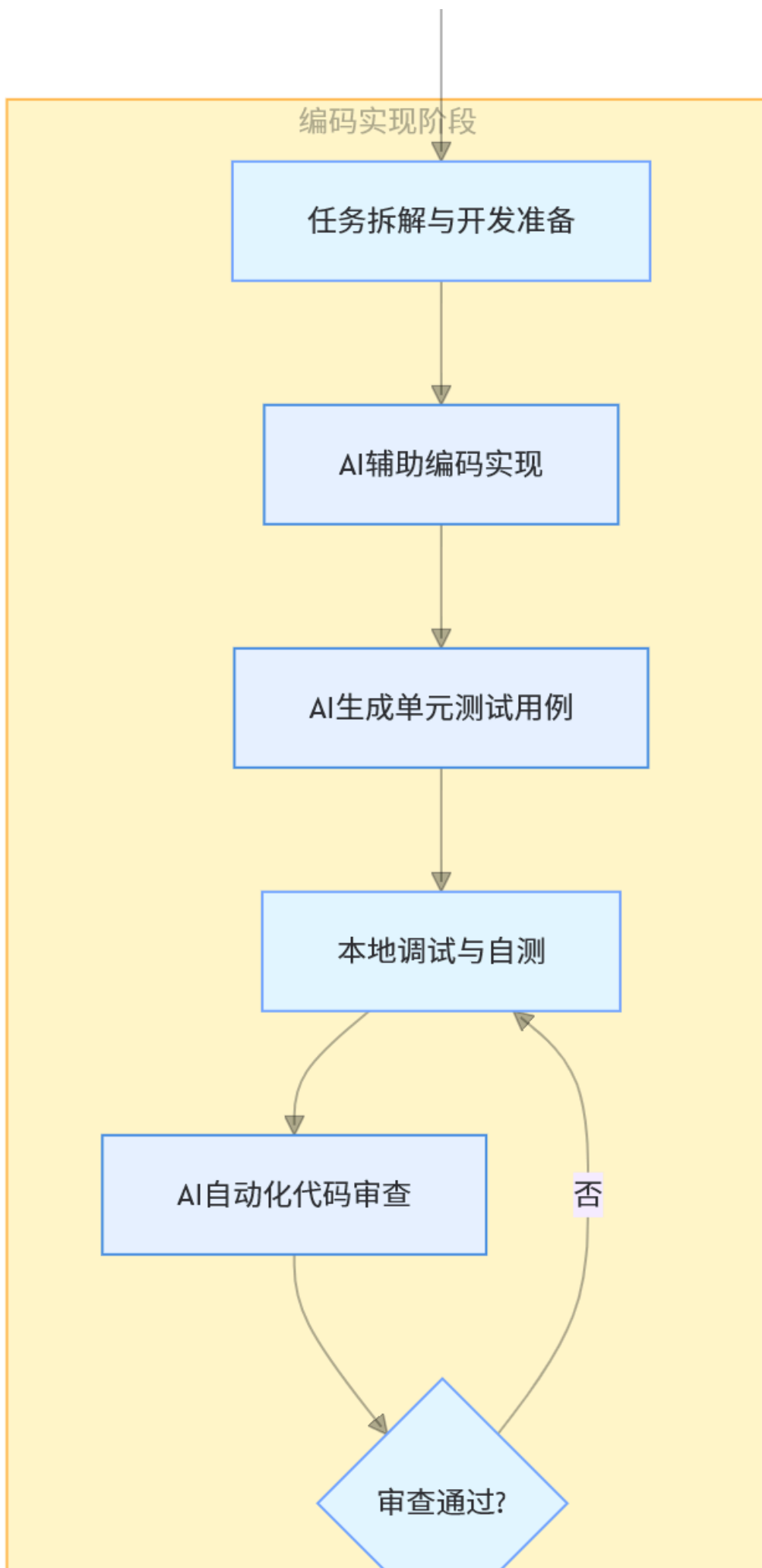
AI生成单元测试用例

本地调试与自测

AI自动化代码审查

审查通过?

否



是

人工评审与代码合入

测试验证阶段

测试计划与策略制定

AI生成测试用例初稿

人工校验与场景补充

AI生成/自愈自动化脚本

测试执行与AI缺陷分析

缺陷闭环与测试总结

运维监控阶段

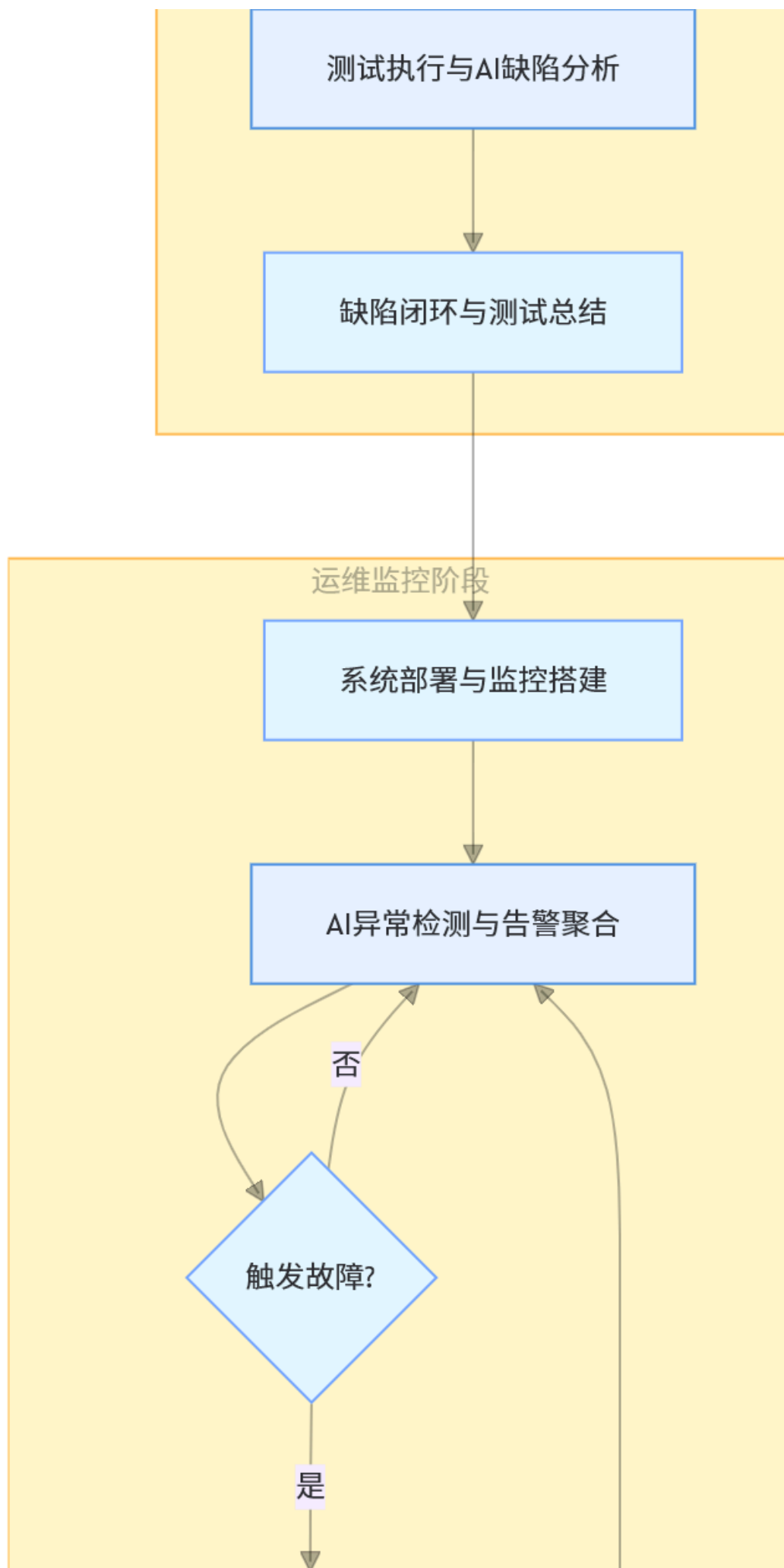
系统部署与监控搭建

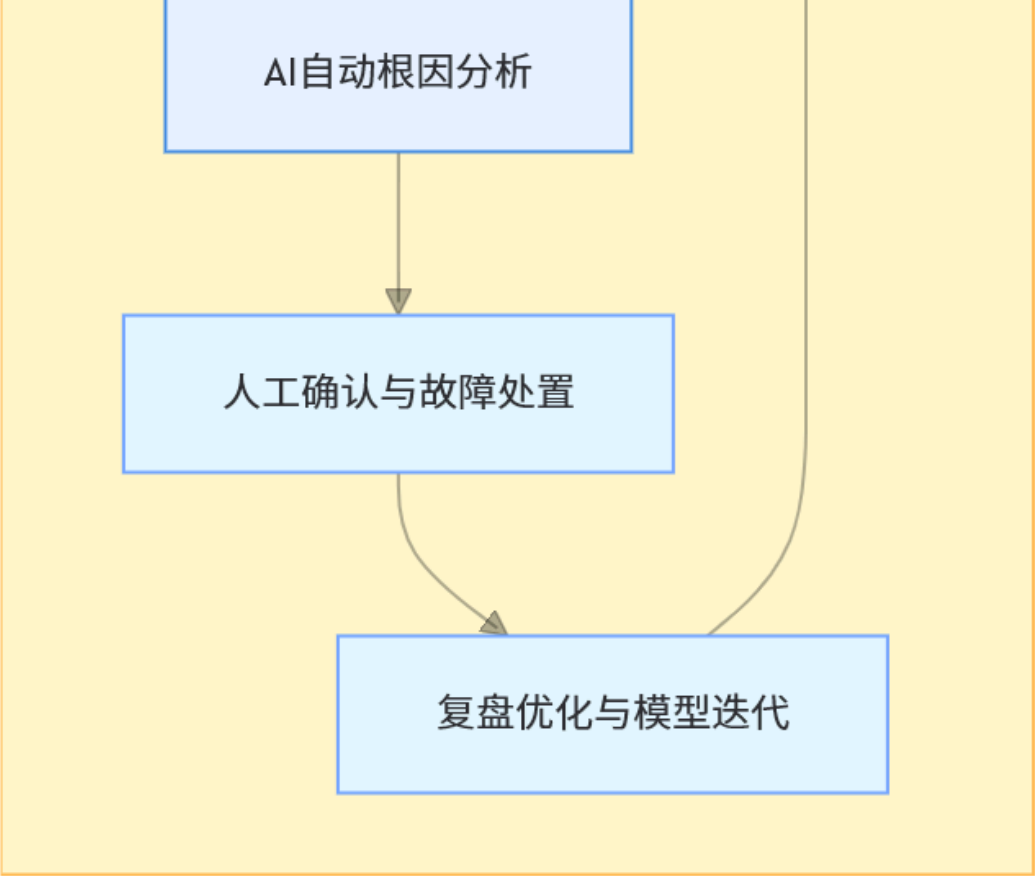
AI异常检测与告警聚合

否

触发故障?

是





注：浅蓝色填充节点为 AI 参与执行的环节，其余为人工主导环节。

## 五、交付物清单与质量标准

### 5.1 全阶段交付物清单

表格

| 阶段   | 交付物名称       | 产出方式           | 负责人      |
|------|-------------|----------------|----------|
| 需求分析 | 需求规格说明书     | AI 生成初稿 + 人工优化 | 需求分析师    |
| 需求分析 | 需求一致性校验报告   | AI 自动生成        | AI 过程赋能师 |
| 系统设计 | 系统架构设计文档    | AI 辅助 + 人工主导   | 架构师      |
| 系统设计 | UML 模型与接口文档 | AI 生成初稿 + 人工优化 | 设计师      |
| 编码实现 | 功能源代码       | 人机协同开发         | 开发工程师    |
| 编码实现 | 代码审查报告      | AI 自动生成        | AI 过程赋能师 |

| 阶段   | 交付物名称     | 产出方式              | 负责人      |
|------|-----------|-------------------|----------|
| 测试验证 | 测试用例集     | AI 生成初稿 + 人工补充    | 测试工程师    |
| 测试验证 | 自动化测试脚本   | AI 生成 / 自愈 + 人工校验 | AI 测试分析师 |
| 测试验证 | 测试总结报告    | 人工撰写 + AI 辅助分析    | 测试工程师    |
| 运维监控 | 告警与根因分析报告 | AI 自动生成           | 运维工程师    |
| 运维监控 | 运维知识库     | 人工沉淀 + AI 整理      | 运维工程师    |

## 5.2 AI 输出物质量标准与验证机制

为保障 AI 输出质量，建立统一的质量准入标准与分级验证机制：

- 1. **需求文档类**：功能需求覆盖率≥95%，术语一致性≥98%，无明显逻辑冲突，文档结构 100% 符合模板规范；执行 “AI 自检 + 人工全量校验” 机制。
- 2. **代码类**：代码规范符合率 100%，无高危安全漏洞，单元测试覆盖率≥80%；执行 “AI 自动扫描 + 核心模块人工全量校验、普通模块抽检” 机制。
- 3. **测试用例类**：需求点覆盖率≥90%，边界与异常场景占比≥30%，有效用例率≥85%；执行 “人工抽检 30%+ 核心场景全量校验” 机制。
- 4. **运维分析类**：根因定位准确率≥85%，告警误报率≤10%；执行 “人工复核确认 + 事后复盘校准” 机制。

# 六、过程度量指标体系

## 6.1 效率类度量指标

- 1. **需求文档产出效率**：单位需求点文档编写耗时；目标值：较传统过程降低 40%。
- 2. **编码生产率**：人均每日产出有效代码行数；目标值：较传统过程提升 35%。
- 3. **测试用例产出效率**：单位需求点测试用例设计耗时；目标值：较传统过程降低 45%。
- 4. **故障平均定位时长**：从告警触发到根因确认的平均耗时；目标值：较传统过程缩短 50%。

## 6.2 质量类度量指标

- 1. **需求返工率**：设计 / 开发阶段因需求问题导致的返工次数占比；目标值：≤10%。
- 2. **代码缺陷密度**：每千行代码 AI 检出 + 人工发现的缺陷数；目标值：≤5 个 / 千行。
- 3. **测试需求覆盖率**：测试用例覆盖的需求点占总需求点比例；目标值：≥95%。
- 4. **线上故障发生率**：每迭代周期线上 P1/P2 级故障数量；目标值：≤2 个 / 迭代。

## 6.3 AI 贡献度专项指标



1. **AI 辅助工作量占比**：AI 辅助完成的工作量占总工作量比例；目标值： $\geq 30\%$ 。
2. **AI 工具覆盖率**：使用 AI 工具的核心环节占比；目标值： $\geq 80\%$ 。
3. **AI 输出一次通过率**：AI 输出内容无需修改直接通过校验的比例；目标值： $\geq 70\%$ 。
4. **AI 驱动效率提升率**：AI 引入后整体环节效率提升比例；目标值： $\geq 30\%$ 。

## 6.4 度量执行机制

每个迭代周期结束后，由项目经理牵头收集过程度量数据，形成迭代过程度量报告，评估过程运行效果与 AI 应用价值，识别改进点。AI 贡献度指标由 AI 过程赋能师负责统计与分析。

# 七、过程监控与持续改进

## 7.1 过程监控机制

建立三级过程监控体系：

1. **日常监控**：各角色每日跟踪任务进度与 AI 工具使用情况，及时反馈异常问题；
2. **迭代回顾**：每个迭代周期结束后开展迭代回顾会，复盘过程执行情况、质量数据与效率表现，识别过程问题；
3. **阶段评审**：项目关键节点开展过程合规性评审，检查过程执行规范性与交付物质量。

## 7.2 AI 能力迭代机制

建立 AI 能力持续优化闭环：

1. 持续沉淀业务领域知识库、Prompt 模板库、最佳实践案例，优化 AI 输出的业务匹配度；
2. 定期跟踪 AI 工具版本更新与行业新技术，评估引入价值，持续升级工具链；
3. 基于质量校验数据与缺陷复盘结果，持续调整 AI 工具参数、生成规则与校验阈值，提升 AI 输出质量。

## 7.3 过程改进流程

过程改进遵循“问题识别→方案制定→试点验证→全量推广”的标准化流程：

1. 基于度量数据与复盘结论识别过程痛点；
2. 由 AI 过程赋能师牵头制定改进方案，明确改进目标与落地路径；
3. 选取试点项目验证改进效果，验证通过后更新过程文档并全量推广；
4. 跟踪改进效果，持续迭代优化过程体系。

# 八、附则

1. 本文档为项目过程执行的唯一依据，所有项目成员需严格按照文档要求开展工作；

2. 本文档的修订需经过程评审通过，由项目经理审批后发布；
3. 本文档解释权归项目过程管理组所有。